Guide DevOps - Mise en Production CNSS

Contents

Guide DevOps — Mise en Production CNSS

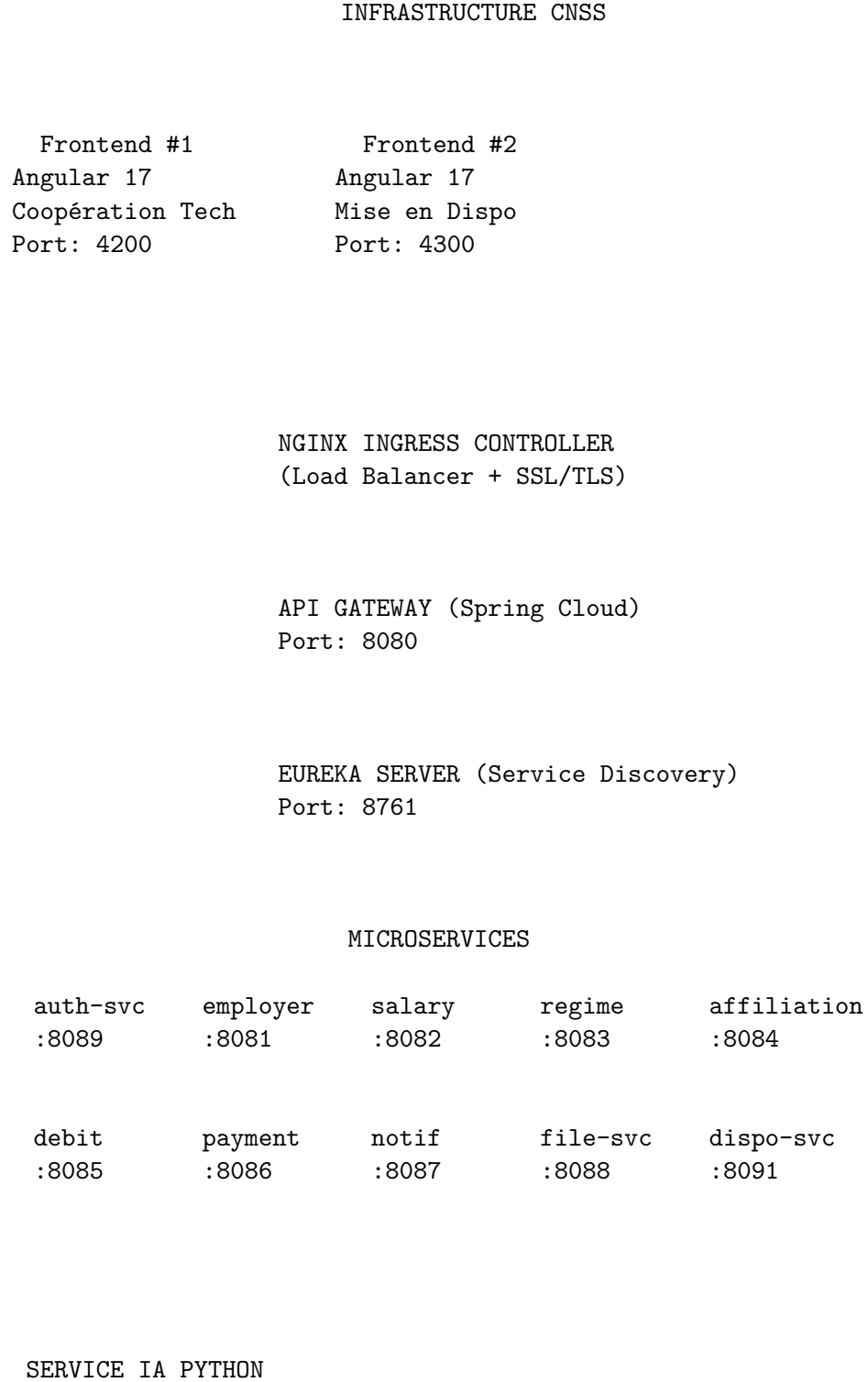
Architecture Microservices Spring Boot + Angular + Python (IA)

Application : Système de Gestion CNSS — Coopération Technique + Mise en Disponibilité Spéciale
Stack : Spring Boot 3.2 · Angular 17 · Python FastAPI · Docker · Kubernetes
Date : Mars 2026

Table des matières

1. Vue d’ensemble de l’architecture

1.1 Composants de l’application CNSS



```
ai-extraction-service (FastAPI + Tesseract OCR) - Port: 8090



INFRASTRUCTURE

 Oracle DB    RabbitMQ     Redis        Volumes (Documents)
 :1521        :5672        :6379        /data/cnss/*
```

1.2 Liste des services à déployer

#	Service	Technologie	Port	Image Docker
1	eureka-server	Spring Boot	8761	cnss/eureka:latest
2	gateway-service	Spring Cloud Gateway	8080	cnss/gateway:latest
3	auth-service	Spring Boot	8089	cnss/auth:latest
4	employer-service	Spring Boot	8081	cnss/employer:latest
5	salary-service	Spring Boot	8082	cnss/salary:latest
6	regime-service	Spring Boot	8083	cnss/regime:latest
7	affiliation-service	Spring Boot	8084	cnss/affiliation:latest
8	debit-service	Spring Boot	8085	cnss/debit:latest
9	payment-service	Spring Boot	8086	cnss/payment:latest
10	notification-service	Spring Boot	8087	cnss/notification:latest
11	file-service	Spring Boot	8088	cnss/file:latest
12	disponibilite-service	Spring Boot	8091	cnss/disponibilite:latest
13	ai-extraction-service	Python FastAPI	8090	cnss/ai-extraction:latest
14	frontend-cooperation	Angular (NGINX)	80	cnss/frontend-coop:latest
15	frontend-disponibilite	Angular (NGINX)	80	cnss/frontend-dispo:latest

---

2. Outils d’Intégration Continue (CI/CD)

2.1 Comparatif des outils CI/CD

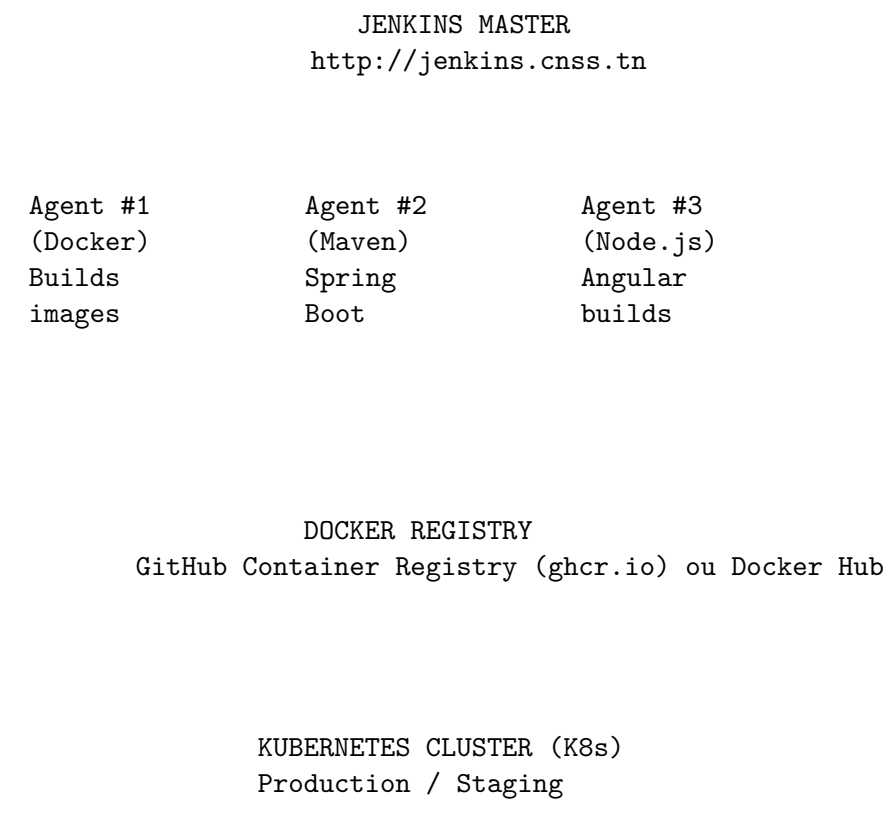
Outil	Points forts	Points faibles	Idéal pour
Jenkins	Très extensible, plugins nombreux, gratuit	Configuration complexe, UI vieillissante	Entreprises avec besoins spécifiques
GitHub Actions	Intégré à GitHub, YAML simple, marketplace riche, runners gratuits	Minutes limitées (plan gratuit)	Projets GitHub, équipes modernes
GitLab CI/CD	Intégré au dépôt, runners distribués	Nécessite GitLab	Équipes utilisant GitLab
Azure DevOps	Intégration Azure/MS parfaite	Vendor lock-in	Environnements Microsoft

2.2 Recommandation pour CNSS : GitHub Actions + Jenkins

Pourquoi Jenkins ?

1. **Gratuit et open source** — pas de coûts de licence
2. **Plugins riches** — intégration native avec Docker, Kubernetes, SonarQube
3. **Pipelines as Code** — Jenkinsfile versionné avec le code
4. **Agents distribués** — parallélisation des builds
5. **Largement adopté** — documentation et support communautaire abondants

Architecture Jenkins pour CNSS



2.3 GitHub Actions — Pipeline CI/CD

Pourquoi GitHub Actions ?

1. **Intégré à GitHub** — pas de serveur séparé à gérer
2. **YAML simple** — fichiers `.github/workflows/*.yml`
3. **Marketplace riche** — actions prêtes à l'emploi
4. **Runners gratuits** — 2000 minutes/mois (plan gratuit)
5. **Self-hosted runners** — possibilité d'utiliser ses propres serveurs

Workflow complet CNSS — `.github/workflows/ci-cd.yml`

```yaml
# .github/workflows/ci-cd.yml
name: CNSS CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_PREFIX: ${{ github.repository_owner }}/cnss

jobs:
  #
  # JOB 1 : BUILD & TEST BACKEND (Spring Boot)
  #
  build-backend:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [auth-service, employer-service, salary-service, regime-service,
                  affiliation-service, debit-service, payment-service,
                  notification-service, file-service, disponibilite-service,
                  eureka-server, gateway-service]
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: gradle
```

```yaml
    - name: Build & Test ${{ matrix.service }}
      working-directory: ${{ matrix.service }}
      run: |
        chmod +x gradlew
        ./gradlew clean build test

    - name: Upload test results
      uses: actions/upload-artifact@v4
      if: always()
      with:
        name: test-results-${{ matrix.service }}
        path: ${{ matrix.service }}/build/test-results/

#
# JOB 2 : BUILD & TEST FRONTEND (Angular)
#
build-frontend:
  runs-on: ubuntu-latest
  strategy:
    matrix:
      app: [frontend, disponibilite-frontend]
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '20'
        cache: 'npm'
        cache-dependency-path: ${{ matrix.app }}/package-lock.json

    - name: Install dependencies
      working-directory: ${{ matrix.app }}
      run: npm ci

    - name: Lint
      working-directory: ${{ matrix.app }}
      run: npm run lint || true

    - name: Build production
      working-directory: ${{ matrix.app }}
      run: npm run build -- --configuration=production

    - name: Upload build artifacts
      uses: actions/upload-artifact@v4
      with:
        name: dist-${{ matrix.app }}
        path: ${{ matrix.app }}/dist/
```

```yaml
#
# JOB 3 : BUILD SERVICE IA (Python)
#
build-ai-service:
  runs-on: ubuntu-latest
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'
        cache: 'pip'

    - name: Install dependencies
      working-directory: ai-extraction-service
      run: |
        pip install -r requirements.txt
        pip install pytest

    - name: Run tests
      working-directory: ai-extraction-service
      run: python -m pytest tests/ -v || true

#
# JOB 4 : SONARQUBE ANALYSIS
#
sonarqube:
  runs-on: ubuntu-latest
  needs: [build-backend, build-frontend]
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
      with:
        fetch-depth: 0

    - name: SonarQube Scan
      uses: SonarSource/sonarqube-scan-action@master
      env:
        SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
        SONAR_HOST_URL: ${{ secrets.SONAR_HOST_URL }}

#
# JOB 5 : BUILD & PUSH DOCKER IMAGES
#
docker-build-push:
  runs-on: ubuntu-latest
  needs: [build-backend, build-frontend, build-ai-service]
  if: github.ref == 'refs/heads/main' || github.ref == 'refs/heads/develop'
```

```yaml
    permissions:
      contents: read
      packages: write
    strategy:
      matrix:
        include:
          - service: auth-service
            context: ./auth-service
          - service: employer-service
            context: ./employer-service
          - service: salary-service
            context: ./salary-service
          - service: disponibilite-service
            context: ./disponibilite-service
          - service: gateway-service
            context: ./gateway-service
          - service: eureka-server
            context: ./eureka-server
          - service: ai-extraction-service
            context: ./ai-extraction-service
          - service: frontend-coop
            context: ./frontend
          - service: frontend-dispo
            context: ./disponibilite-frontend
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Extract metadata
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.REGISTRY }}/${{ env.IMAGE_PREFIX }}/${{ matrix.service }}
          tags: |
            type=sha,prefix=
            type=ref,event=branch
            type=raw,value=latest,enable=${{ github.ref == 'refs/heads/main' }}

      - name: Build and push
        uses: docker/build-push-action@v5
```

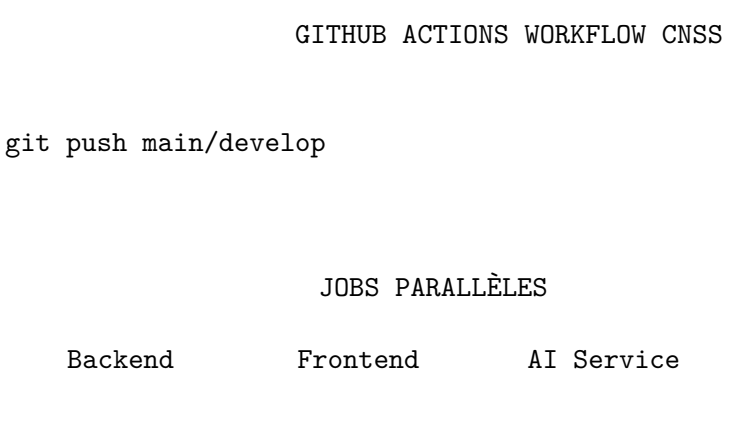
```yaml
      with:
        context: ${{ matrix.context }}
        push: true
        tags: ${{ steps.meta.outputs.tags }}
        labels: ${{ steps.meta.outputs.labels }}
        cache-from: type=gha
        cache-to: type=gha,mode=max

#
# JOB 6 : DEPLOY TO KUBERNETES (Staging)
#
deploy-staging:
  runs-on: ubuntu-latest
  needs: docker-build-push
  if: github.ref == 'refs/heads/develop'
  environment: staging
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up kubectl
      uses: azure/setup-kubectl@v3

    - name: Configure kubeconfig
      run: |
        mkdir -p ~/.kube
        echo "${{ secrets.KUBECONFIG_STAGING }}" | base64 -d > ~/.kube/config

    - name: Deploy with Helm
      run: |
        helm upgrade --install cnss-staging ./helm/cnss \
          --namespace cnss-staging \
          --create-namespace \
          --set image.tag=${{ github.sha }} \
          --set environment=staging \
          -f ./helm/cnss/values-staging.yaml

#
# JOB 7 : DEPLOY TO KUBERNETES (Production)
#
deploy-production:
  runs-on: ubuntu-latest
  needs: docker-build-push
  if: github.ref == 'refs/heads/main'
  environment: production
  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Set up kubectl
```

```
      uses: azure/setup-kubectl@v3

    - name: Configure kubeconfig
      run: |
        mkdir -p ~/.kube
        echo "${{ secrets.KUBECONFIG_PROD }}" | base64 -d > ~/.kube/config

    - name: Deploy with Helm
      run: |
        helm upgrade --install cnss-prod ./helm/cnss \
          --namespace cnss-prod \
          --create-namespace \
          --set image.tag=${{ github.sha }} \
          --set environment=production \
          -f ./helm/cnss/values-prod.yaml

    - name: Notify Slack
      uses: slackapi/slack-github-action@v1
      with:
        payload: |
          {
            "text": "  CNSS deployed to production! Commit: ${{ github.sha }}"
          }
      env:
        SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}
```

Secrets GitHub à configurer

Secret	Description
SONAR_TOKEN	Token d'authentification SonarQube
SONAR_HOST_URL	URL du serveur SonarQube
KUBECONFIG_STAGING	Kubeconfig encodé base64 (staging)
KUBECONFIG_PROD	Kubeconfig encodé base64 (production)
SLACK_WEBHOOK_URL	Webhook Slack pour notifications

Schéma du pipeline GitHub Actions



```
   (12 svcs)      (2 apps)       (Python)
   matrix         matrix


                  SONARQUBE ANALYSIS


              DOCKER BUILD & PUSH (ghcr.io)
              15 images en parallèle (matrix)


 deploy-staging            deploy-production
 (branch develop)          (branch main)
 auto                      approval requis
```

2.4 Installation de Jenkins

Option 1 : Docker Compose (Développement/Test)

```yaml
# jenkins-docker-compose.yml
version: '3.8'

services:
  jenkins:
    image: jenkins/jenkins:lts-jdk17
    container_name: jenkins
    restart: always
    ports:
      - "8082:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock
    environment:
      - JAVA_OPTS=-Xmx2g -Xms512m
    networks:
      - cnss-network
```

```yaml
  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    restart: always
    ports:
      - "9000:9000"
    environment:
      - SONAR_JDBC_URL=jdbc:postgresql://sonar-db:5432/sonar
      - SONAR_JDBC_USERNAME=sonar
      - SONAR_JDBC_PASSWORD=sonar
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_logs:/opt/sonarqube/logs
    depends_on:
      - sonar-db
    networks:
      - cnss-network

  sonar-db:
    image: postgres:15-alpine
    container_name: sonar-db
    restart: always
    environment:
      - POSTGRES_USER=sonar
      - POSTGRES_PASSWORD=sonar
      - POSTGRES_DB=sonar
    volumes:
      - sonar_db_data:/var/lib/postgresql/data
    networks:
      - cnss-network

volumes:
  jenkins_home:
  sonarqube_data:
  sonarqube_logs:
  sonar_db_data:

networks:
  cnss-network:
    external: true
```

Option 2 : Installation sur serveur dédié

```bash
# Installation Jenkins sur Ubuntu 22.04
sudo apt update
sudo apt install -y openjdk-17-jdk

# Ajouter le dépôt Jenkins
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null
```

```
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt update
sudo apt install -y jenkins

# Démarrer Jenkins
sudo systemctl enable jenkins
sudo systemctl start jenkins

# Récupérer le mot de passe initial
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

2.4 Plugins Jenkins recommandés

Plugin	Fonction
Docker Pipeline	Build et push d'images Docker
Kubernetes	Déploiement sur K8s
SonarQube Scanner	Analyse qualité du code
Pipeline	Pipelines as Code (Jenkinsfile)
Blue Ocean	Interface moderne pour pipelines
Git	Intégration avec Git/GitHub
Credentials Binding	Gestion sécurisée des secrets
Email Extension	Notifications par email
Slack Notification	Notifications Slack

---

3. Outils d'Orchestration

3.1 Comparatif des solutions d'orchestration

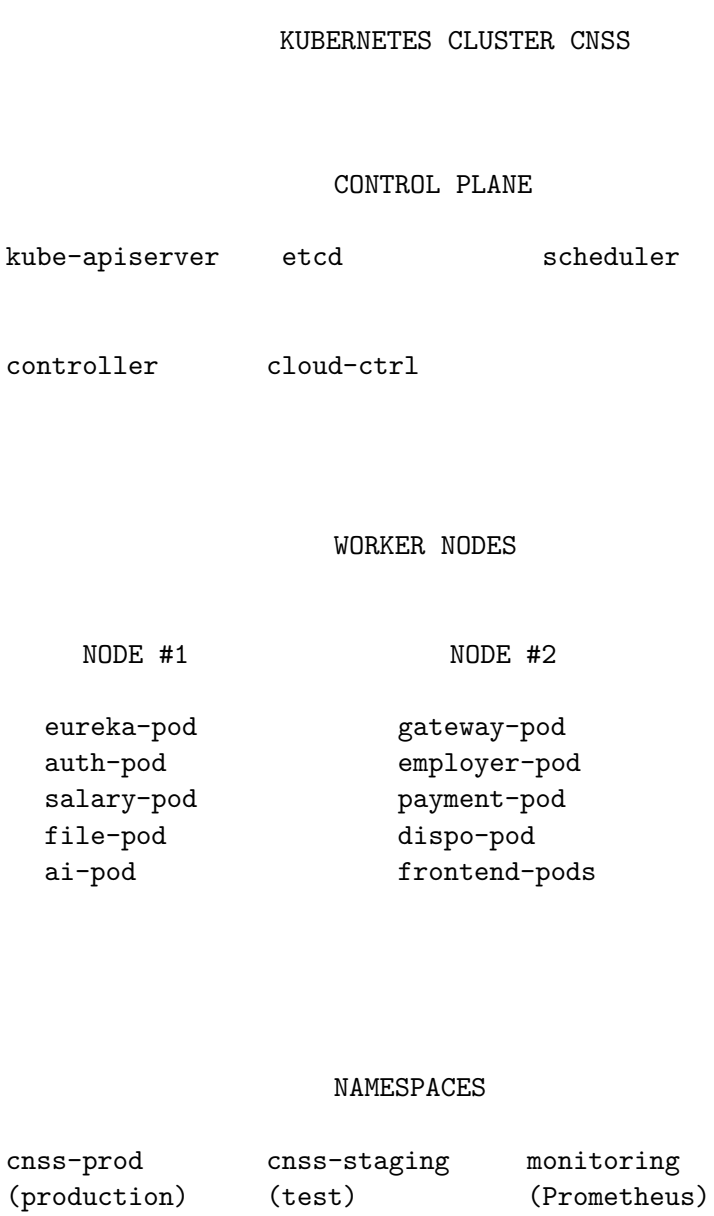
Solution	Avantages	Inconvénients	Coût
Kubernetes (K8s)	Standard industrie, scalable, résilient	Complexe à configurer	Gratuit (infra à charge)
Docker Swarm	Simple, intégré à Docker	Moins de fonctionnalités	Gratuit
Portainer	UI intuitive, gère Docker/K8s	Limité pour gros clusters	Freemium
OpenShift	K8s + sécurité + CI/CD intégrés	Coûteux, lourd	Licence Red Hat
Rancher	Multi-cluster K8s, UI simple	Ressources supplémentaires	Gratuit

3.2 Recommandation : Kubernetes + Portainer

Pourquoi cette combinaison ?

- **Kubernetes** : orchestration robuste, scaling automatique, self-healing
- **Portainer** : interface graphique pour gérer K8s facilement (idéal pour équipes non-DevOps)

Architecture Kubernetes pour CNSS



3.3 Installation de Kubernetes (k3s — léger)

Pour un environnement CNSS avec ressources limitées, **k3s** est recommandé :

```
# Installation k3s sur le serveur master
curl -sfL https://get.k3s.io | sh -

# Vérifier l'installation
sudo k3s kubectl get nodes

# Récupérer le token pour les workers
sudo cat /var/lib/rancher/k3s/server/node-token

# Sur les workers (remplacer IP et TOKEN)
curl -sfL https://get.k3s.io | K3S_URL=https://<MASTER_IP>:6443 K3S_TOKEN=<TOKEN> sh -
```

3.4 Installation de Portainer

```
# Installer Portainer sur Kubernetes
kubectl apply -n portainer -f https://raw.githubusercontent.com/portainer/k8s/master/deploy/manif

# Exposer Portainer via NodePort
kubectl patch svc portainer -n portainer -p '{"spec": {"type": "NodePort", "ports": [{"port": 900

# Accéder à Portainer : http://<NODE_IP>:30777
```

3.5 Helm — Gestionnaire de packages Kubernetes

```
# Installation Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash

# Ajouter des repositories utiles
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

# Exemple : installer Redis via Helm
helm install redis bitnami/redis -n cnss-prod
```

---

4. Pipeline CI/CD Complet

4.1 Jenkinsfile — Pipeline complet CNSS

```
// Jenkinsfile - Pipeline CI/CD CNSS
pipeline {
    agent any

    environment {
```

```
    DOCKER_REGISTRY = 'registry.cnss.tn'
    DOCKER_CREDENTIALS = credentials('docker-registry-creds')
    SONAR_TOKEN = credentials('sonarqube-token')
    KUBECONFIG = credentials('kubeconfig-prod')
    VERSION = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
}

stages {
    //
    // ÉTAPE 1 : CHECKOUT DU CODE SOURCE
    //
    stage('Checkout') {
        steps {
            checkout scm
            script {
                env.GIT_COMMIT = sh(script: 'git rev-parse HEAD', returnStdout: true).trim()
                env.GIT_BRANCH = sh(script: 'git rev-parse --abbrev-ref HEAD', returnStdout:
            }
        }
    }

    //
    // ÉTAPE 2 : BUILD & TEST - BACKEND SPRING BOOT
    //
    stage('Build Backend Services') {
        parallel {
            stage('Auth Service') {
                steps {
                    dir('auth-service') {
                        sh './gradlew clean build -x test'
                        sh './gradlew test'
                    }
                }
            }
            stage('Employer Service') {
                steps {
                    dir('employer-service') {
                        sh './gradlew clean build -x test'
                        sh './gradlew test'
                    }
                }
            }
            stage('Salary Service') {
                steps {
                    dir('salary-service') {
                        sh './gradlew clean build -x test'
                        sh './gradlew test'
                    }
                }
            }
```

```
        stage('Disponibilite Service') {
            steps {
                dir('disponibilite-service') {
                    sh './gradlew clean build -x test'
                    sh './gradlew test'
                }
            }
        }
        stage('AI Extraction Service') {
            steps {
                dir('ai-extraction-service') {
                    sh 'pip install -r requirements.txt'
                    sh 'python -m pytest tests/ --junitxml=test-results.xml || true'
                }
            }
        }
    }
}

//
// ÉTAPE 3 : BUILD & TEST - FRONTEND ANGULAR
//
stage('Build Frontend') {
    parallel {
        stage('Frontend Coopération') {
            steps {
                dir('frontend') {
                    sh 'npm ci'
                    sh 'npm run lint || true'
                    sh 'npm run test -- --watch=false --browsers=ChromeHeadless || true'
                    sh 'npm run build -- --configuration=production'
                }
            }
        }
        stage('Frontend Disponibilité') {
            steps {
                dir('disponibilite-frontend') {
                    sh 'npm ci'
                    sh 'npm run lint || true'
                    sh 'npm run build -- --configuration=production'
                }
            }
        }
    }
}

//
// ÉTAPE 4 : ANALYSE QUALITÉ - SONARQUBE
//
stage('SonarQube Analysis') {
```

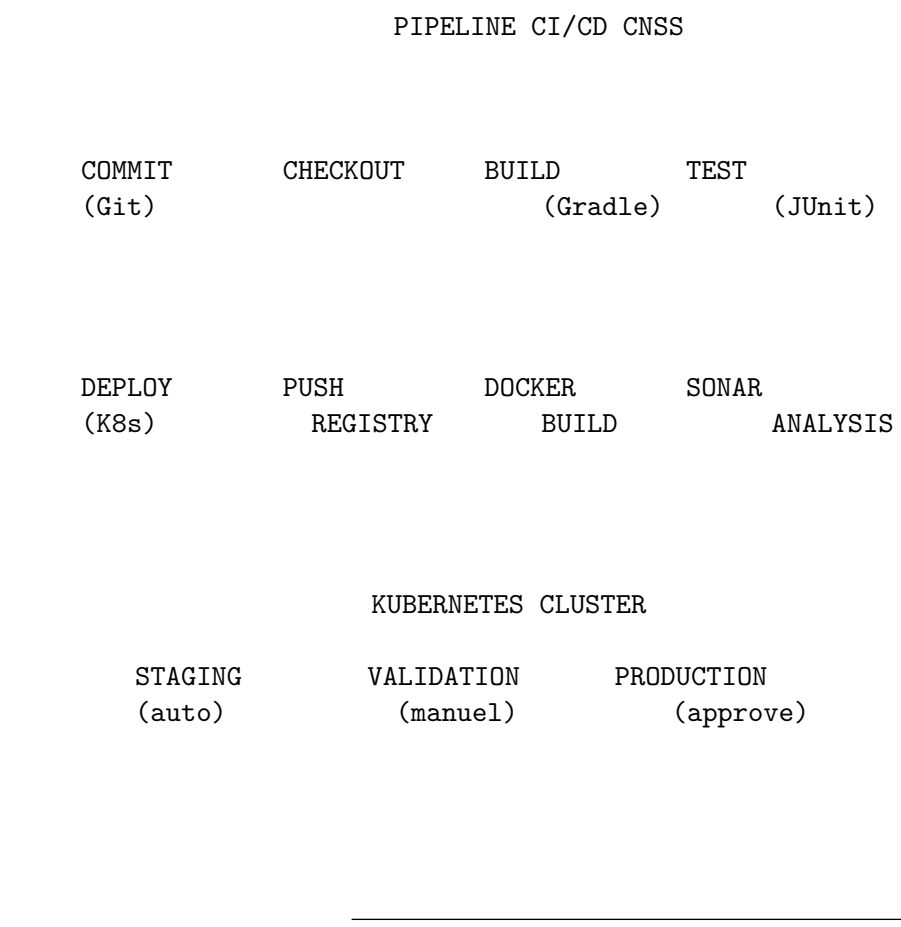
```
        steps {
            withSonarQubeEnv('SonarQube') {
                sh '''
                    sonar-scanner \
                      -Dsonar.projectKey=cnss-cooperation \
                      -Dsonar.projectName="CNSS Cooperation" \
                      -Dsonar.sources=. \
                      -Dsonar.java.binaries=**/build/classes \
                      -Dsonar.exclusions=**/node_modules/**,**/dist/**,**/*.spec.ts
                '''
            }
        }
    }

    stage('Quality Gate') {
        steps {
            timeout(time: 5, unit: 'MINUTES') {
                waitForQualityGate abortPipeline: false
            }
        }
    }

    //
    // ÉTAPE 5 : BUILD IMAGES DOCKER
    //
    stage('Build Docker Images') {
        steps {
            script {
                def services = [
                    'eureka-server',
                    'gateway-service',
                    'auth-service',
                    'employer-service',
                    'salary-service',
                    'regime-service',
                    'affiliation-service',
                    'debit-service',
                    'payment-service',
                    'notification-service',
                    'file-service',
                    'disponibilite-service',
                    'ai-extraction-service'
                ]

                services.each { svc ->
                    dir(svc) {
                        sh "docker build -t ${DOCKER_REGISTRY}/cnss/${svc}:${VERSION} ."
                        sh "docker tag ${DOCKER_REGISTRY}/cnss/${svc}:${VERSION} ${DOCKER_REG
                    }
                }
```

```
                // Frontends
                dir('frontend') {
                    sh "docker build -t ${DOCKER_REGISTRY}/cnss/frontend-coop:${VERSION} ."
                }
                dir('disponibilite-frontend') {
                    sh "docker build -t ${DOCKER_REGISTRY}/cnss/frontend-dispo:${VERSION} ."
                }
            }
        }
    }

    //
    // ÉTAPE 6 : PUSH VERS LE REGISTRE DOCKER
    //
    stage('Push to Registry') {
        steps {
            script {
                docker.withRegistry("https://${DOCKER_REGISTRY}", 'docker-registry-creds') {
                    def services = [
                        'eureka-server', 'gateway-service', 'auth-service',
                        'employer-service', 'salary-service', 'regime-service',
                        'affiliation-service', 'debit-service', 'payment-service',
                        'notification-service', 'file-service', 'disponibilite-service',
                        'ai-extraction-service', 'frontend-coop', 'frontend-dispo'
                    ]

                    services.each { svc ->
                        sh "docker push ${DOCKER_REGISTRY}/cnss/${svc}:${VERSION}"
                        sh "docker push ${DOCKER_REGISTRY}/cnss/${svc}:latest"
                    }
                }
            }
        }
    }

    //
    // ÉTAPE 7 : DÉPLOIEMENT KUBERNETES (STAGING)
    //
    stage('Deploy to Staging') {
        when {
            branch 'develop'
        }
        steps {
            script {
                withKubeConfig([credentialsId: 'kubeconfig-staging']) {
                    sh '''
                        helm upgrade --install cnss-staging ./helm/cnss \
                          --namespace cnss-staging \
                          --create-namespace \
```

```
                        --set image.tag=${VERSION} \
                        --set environment=staging \
                        -f ./helm/cnss/values-staging.yaml
                    '''
                }
            }
        }
    }

    //
    // ÉTAPE 8 : DÉPLOIEMENT KUBERNETES (PRODUCTION)
    //
    stage('Deploy to Production') {
        when {
            branch 'main'
        }
        steps {
            input message: 'Déployer en production ?', ok: 'Déployer'
            script {
                withKubeConfig([credentialsId: 'kubeconfig-prod']) {
                    sh '''
                        helm upgrade --install cnss-prod ./helm/cnss \
                          --namespace cnss-prod \
                          --create-namespace \
                          --set image.tag=${VERSION} \
                          --set environment=production \
                          -f ./helm/cnss/values-prod.yaml
                    '''
                }
            }
        }
    }
}

//
// POST-ACTIONS
//
post {
    success {
        slackSend channel: '#cnss-deployments',
                  color: 'good',
                  message: "  Build #${BUILD_NUMBER} réussi - ${GIT_BRANCH}\nCommit: ${GIT_CON
    }
    failure {
        slackSend channel: '#cnss-deployments',
                  color: 'danger',
                  message: "  Build #${BUILD_NUMBER} échoué - ${GIT_BRANCH}\nCommit: ${GIT_CON
    }
    always {
        cleanWs()
```

```
            sh 'docker system prune -f || true'
        }
    }
}
```

4.2 Schéma du pipeline CI/CD



---

5. Configuration Kubernetes

5.1 Structure des fichiers Helm

```
helm/cnss/
  Chart.yaml
  values.yaml
  values-staging.yaml
  values-prod.yaml
  templates/
      namespace.yaml
      configmap.yaml
      secrets.yaml
      deployments/
          eureka-deployment.yaml
```

```
        gateway-deployment.yaml
        auth-deployment.yaml
        employer-deployment.yaml
        salary-deployment.yaml
        disponibilite-deployment.yaml
        ai-extraction-deployment.yaml
        frontend-deployment.yaml
    services/
        services.yaml
    ingress.yaml
    hpa.yaml  (Horizontal Pod Autoscaler)
```

5.2 Exemple de Deployment Kubernetes

```yaml
# templates/deployments/auth-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: {{ .Values.namespace }}
  labels:
    app: auth-service
    version: {{ .Values.image.tag }}
spec:
  replicas: {{ .Values.auth.replicas }}
  selector:
    matchLabels:
      app: auth-service
  template:
    metadata:
      labels:
        app: auth-service
    spec:
      containers:
        - name: auth-service
          image: "{{ .Values.image.registry }}/cnss/auth-service:{{ .Values.image.tag }}"
          imagePullPolicy: Always
          ports:
            - containerPort: 8089
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: {{ .Values.environment }}
            - name: EUREKA_CLIENT_SERVICEURL_DEFAULTZONE
              value: "http://eureka-server:8761/eureka"
            - name: SPRING_DATASOURCE_URL
              valueFrom:
                secretKeyRef:
                  name: cnss-secrets
                  key: db-url
            - name: SPRING_DATASOURCE_PASSWORD
```

```
            valueFrom:
              secretKeyRef:
                name: cnss-secrets
                key: db-password
          - name: JWT_SECRET
            valueFrom:
              secretKeyRef:
                name: cnss-secrets
                key: jwt-secret
        resources:
          requests:
            memory: "256Mi"
            cpu: "100m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8089
          initialDelaySeconds: 60
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8089
          initialDelaySeconds: 30
          periodSeconds: 5
      imagePullSecrets:
        - name: registry-credentials
```

5.3 Ingress Controller (NGINX)

```
# templates/ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: cnss-ingress
  namespace: {{ .Values.namespace }}
  annotations:
    kubernetes.io/ingress.class: nginx
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "50m"
spec:
  tls:
    - hosts:
        - cooperation.cnss.tn
        - disponibilite.cnss.tn
        - api.cnss.tn
```

```
    secretName: cnss-tls
  rules:
    # Frontend Coopération Technique
    - host: cooperation.cnss.tn
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-coop
                port:
                  number: 80

    # Frontend Mise en Disponibilité
    - host: disponibilite.cnss.tn
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-dispo
                port:
                  number: 80

    # API Gateway
    - host: api.cnss.tn
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: gateway-service
                port:
                  number: 8080
```

5.4 Horizontal Pod Autoscaler

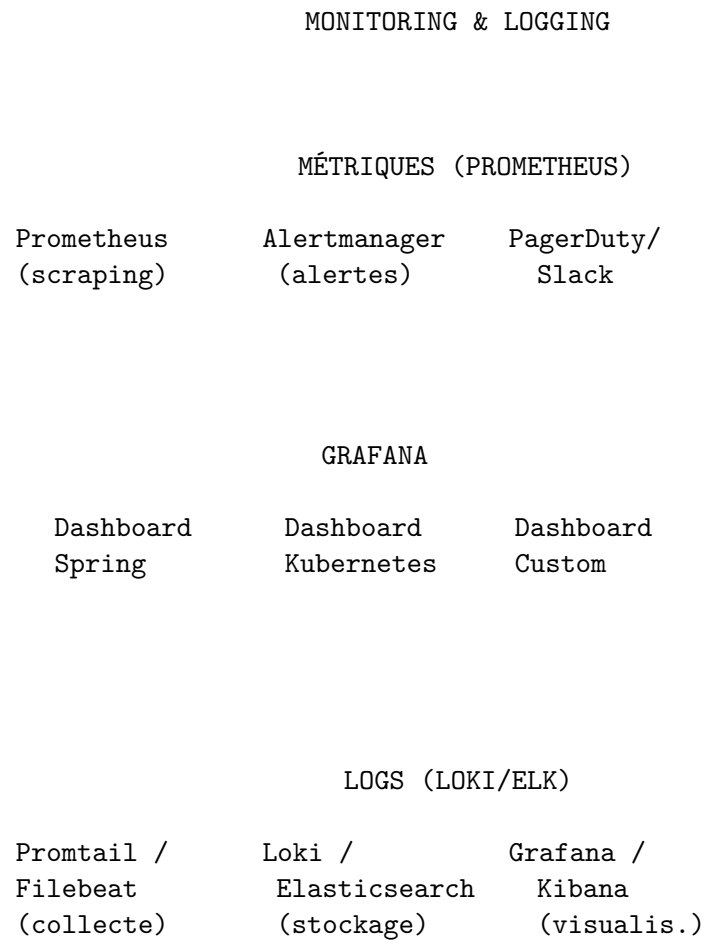
```
# templates/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: gateway-hpa
  namespace: {{ .Values.namespace }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
```

```yaml
  name: gateway-service
minReplicas: 2
maxReplicas: 10
metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
```

---

6. Monitoring et Logging

6.1 Stack de monitoring recommandée



6.2 Installation Prometheus + Grafana via Helm

```
# Installer le stack kube-prometheus
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

helm install prometheus prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --create-namespace \
  --set grafana.adminPassword=SecurePassword123 \
  --set prometheus.prometheusSpec.retention=30d \
  --set alertmanager.enabled=true

# Accéder à Grafana
kubectl port-forward svc/prometheus-grafana -n monitoring 3000:80
# URL: http://localhost:3000 (admin / SecurePassword123)
```

6.3 Installation Loki pour les logs

```
# Installer Loki + Promtail
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update

helm install loki grafana/loki-stack \
  --namespace monitoring \
  --set promtail.enabled=true \
  --set loki.persistence.enabled=true \
  --set loki.persistence.size=50Gi
```

6.4 Métriques Spring Boot (Actuator)

```
# application-prod.yml - Configuration Actuator
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      show-details: when_authorized
      probes:
        enabled: true
  metrics:
    export:
      prometheus:
```

```
        enabled: true
    tags:
      application: ${spring.application.name}
```

---

7. Sécurité en Production

7.1 Checklist sécurité

#	Élément	Action
1	**Secrets**	Utiliser Kubernetes Secrets ou HashiCorp Vault
2	**Images Docker**	Scanner avec Trivy ou Snyk
3	**Registre Docker**	Utiliser un registre privé avec authentification
4	**HTTPS/TLS**	Certificats Let's Encrypt via cert-manager
5	**RBAC**	Limiter les permissions Kubernetes par namespace
6	**Network Policies**	Restreindre le trafic entre pods
7	**Pod Security**	Activer Pod Security Standards
8	**Mises à jour**	Automatiser les patches de sécurité

7.2 Scanning des images Docker

```
# Installation Trivy
sudo apt install trivy

# Scanner une image
trivy image registry.cnss.tn/cnss/auth-service:latest

# Intégrer dans le pipeline Jenkins
stage('Security Scan') {
    steps {
        sh 'trivy image --exit-code 1 --severity HIGH,CRITICAL ${IMAGE_NAME}'
    }
}
```

7.3 Network Policies

```
# Autoriser uniquement le trafic depuis le gateway vers les services
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: cnss-prod
```

```yaml
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
    - Ingress
  ingress:
    - from:
        - podSelector:
            matchLabels:
              app: gateway-service
      ports:
        - protocol: TCP
          port: 8080
```

---

8. Comparatif des Solutions Cloud

8.1 Options de déploiement

Solution	Coût mensuel (estimé)	Complexité	Idéal pour
On-Premise (k3s)	Coût serveurs uniquement	Moyen	Budget limité, contrôle total
AWS EKS	~150-500$/mois \| Faible \| Scalabilité, intégrations AWS \| \| ** Azure AKS ** \| 100−400$/mois	Faible	Environnement Microsoft
Google GKE	~100-400$/mois \| Faible \| Performance, ML/IA \| \| ** DigitalOcean DOKS ** \| 50 − 200$/mois	Très faible	Simplicité, startups
OVHcloud Managed K8s	~80-300$/mois	Faible	Souveraineté données (EU)

8.2 Recommandation pour CNSS (Tunisie)

Option 1 : Infrastructure locale (recommandée pour administration publique) - 3 serveurs bare-metal ou VM (16 Go RAM, 4 vCPU chacun) - k3s ou Rancher pour l'orchestration - Portainer pour la gestion UI - Coût : uniquement matériel + électricité

Option 2 : Cloud hybride - Kubernetes managé (OVHcloud ou Scaleway — datacenters EU/proches) - Base de données Oracle reste on-premise - VPN site-to-site pour la connexion

---

9. Mise en œuvre pas à pas

Phase 1 : Préparation (1 semaine)

Jour	Tâche
J1-J2	Installer Jenkins + SonarQube (Docker Compose)
J3-J4	Configurer le registre Docker privé
J5	Écrire le Jenkinsfile de base

Phase 2 : Kubernetes (1 semaine)

Jour	Tâche
J6-J7	Installer k3s sur les serveurs
J8	Installer Portainer
J9-J10	Créer les manifests/Helm charts

Phase 3 : Pipeline CI/CD (1 semaine)

Jour	Tâche
J11-J12	Configurer le pipeline Jenkins complet
J13	Tester le déploiement staging
J14-J15	Configurer les alertes et notifications

Phase 4 : Monitoring (3 jours)

Jour	Tâche
J16	Installer Prometheus + Grafana
J17	Installer Loki pour les logs
J18	Créer les dashboards personnalisés

Phase 5 : Production (2 jours)

Jour	Tâche
J19	Déploiement production
J20	Tests de charge, documentation finale

□ Résumé des outils recommandés pour CNSS

Catégorie	Outil principal	Alternative
CI/CD	GitHub Actions	Jenkins (alternative)
Qualité code	SonarQube	—

Catégorie	Outil principal	Alternative
Registre Docker	GitHub Container Registry (ghcr.io)	Docker Hub / Harbor
Orchestration	Kubernetes (k3s)	Docker Swarm
Gestion K8s	Portainer	Rancher
Déploiement	Helm + ArgoCD	kubectl apply
Monitoring	Prometheus + Grafana	Datadog (payant)
Logs	Loki + Grafana	ELK Stack
Sécurité images	Trivy	Snyk
Certificats SSL	cert-manager + Let’s Encrypt	—

Document préparé pour le projet CNSS — Coopération Technique et Mise en Disponibilité Spéciale
Date : Mars 2026
Version : 1.0